Introduction to Artificial Intelligence

Chapter 5 – Part I: What CSP is and constraint graph

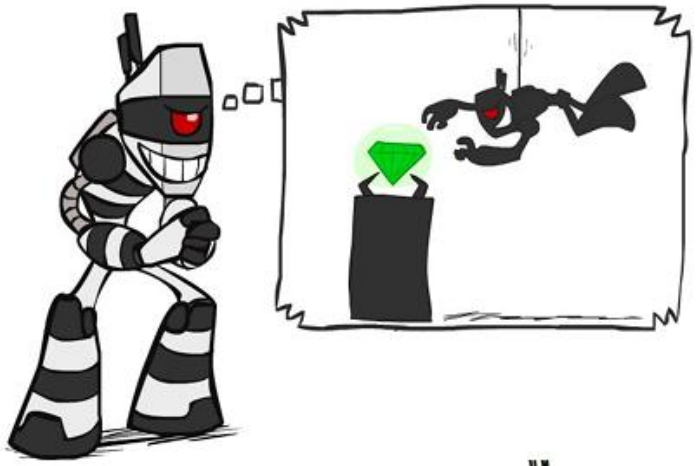
Soongsil University



Source: UC Berkeley CS188, slides by Dan Klein and Pieter Abbeel (ai.berkeley.edu)

What is Search For?

- **ASSUMPTIONS** about the world
  - EX) The ordinary vacuum world: a single agent, deterministic actions, fully observed state, discrete state space.



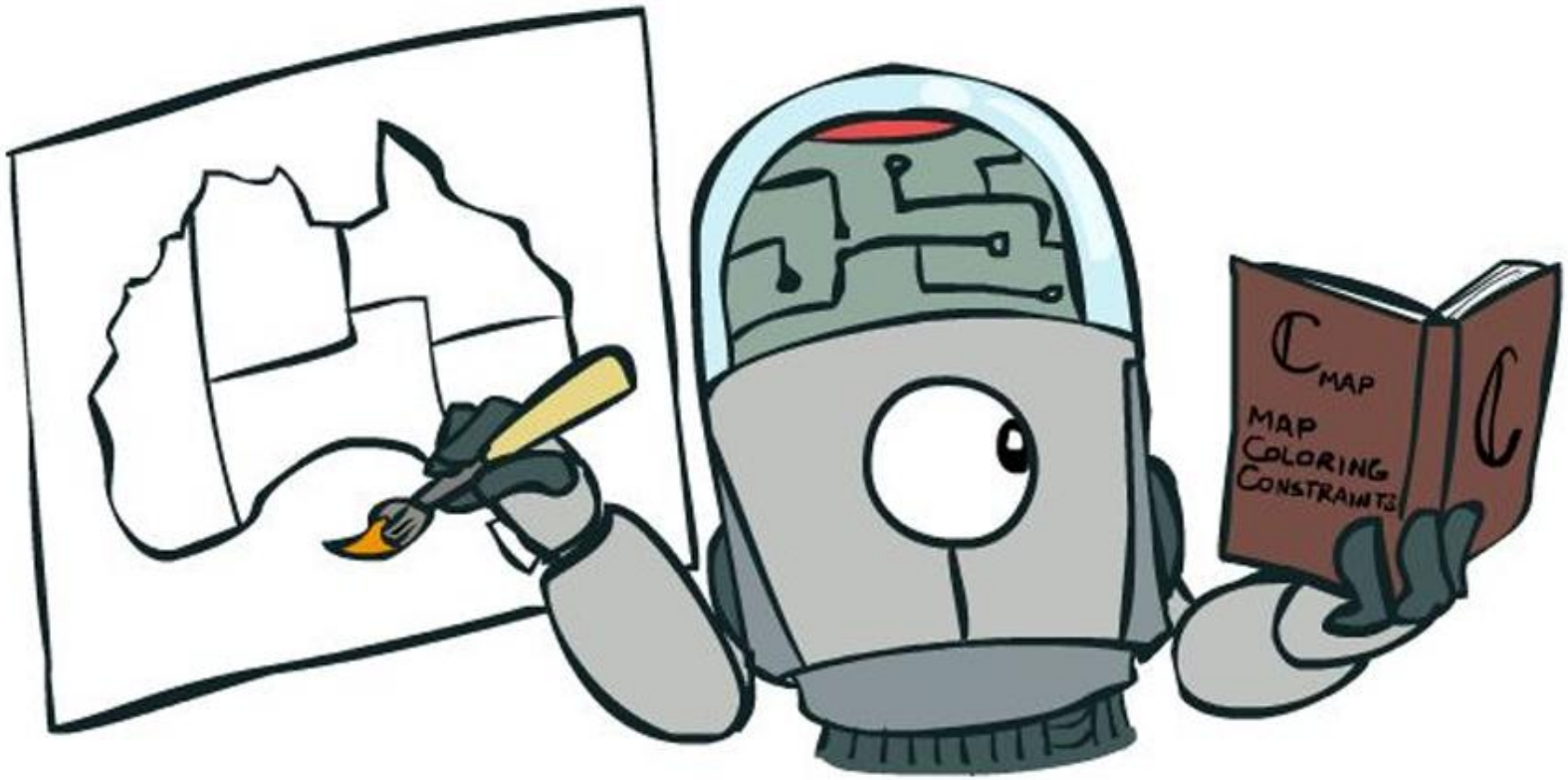
- **PLANNING** problem: a sequence of actions
  - The path is the most important.
  - Paths have costs and depths.
  - Heuristics give problem-specific guidance.



- **IDENTIFICATION** problem: assignment to variables.
  - The goal itself is what matters, not how you got there
  - All paths at the same depth (for some formulations)
  - CSPs are a specialized class of identification problem

1. Constraint Satisfaction Problems

- What a CSP is, and what makes it different from ordinary search.
  - A standard search problem: just a start state, and then this function that let you explore outward. They have only Action, Results, Is-GOAL.
  - A CSP: variables, domains, and a goal test you can read. (**structure of the state**)



Constraint Satisfaction Problems

- Standard search problem:
  - State is a **BLACK BOX**: an arbitrary data structure.
  - Goal test can be any function over state
  - Successor function can also be anything

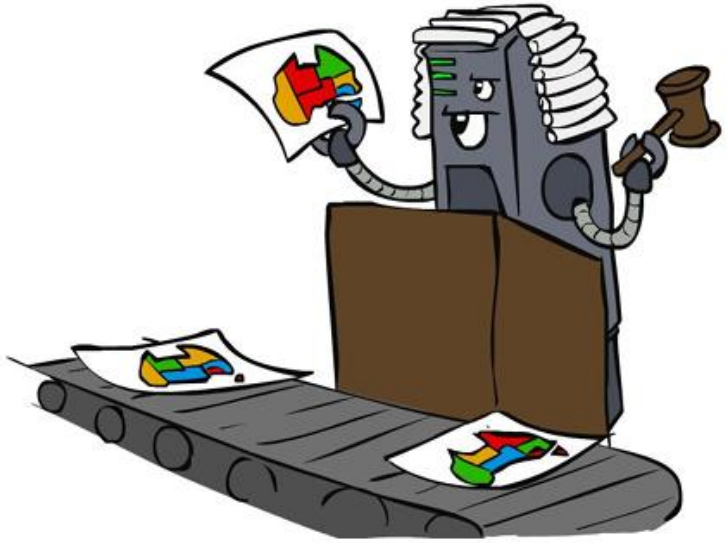
```
# ——  Here we see the state ——
def ACTIONS(s):      ...   # 상태 1이면 [Suck, Right, Left]
def RESULT(s, a):    ...   # RESULT(1, Suck) = 5
def IS_GOAL(s):      return s in (7, 8)

# —— Search Code: BFS (State is black box) ——
def BFS(problem):
    frontier = [problem.INITIAL]
    reached  = {problem.INITIAL}
    while frontier:
        s = frontier.pop(0)
        if problem.IS_GOAL(s): return s
        for a in problem.ACTIONS(s):
            s2 = problem.RESULT(s, a)
            if s2 not in reached:
                reached.add(s2); frontier.append(s2)
```

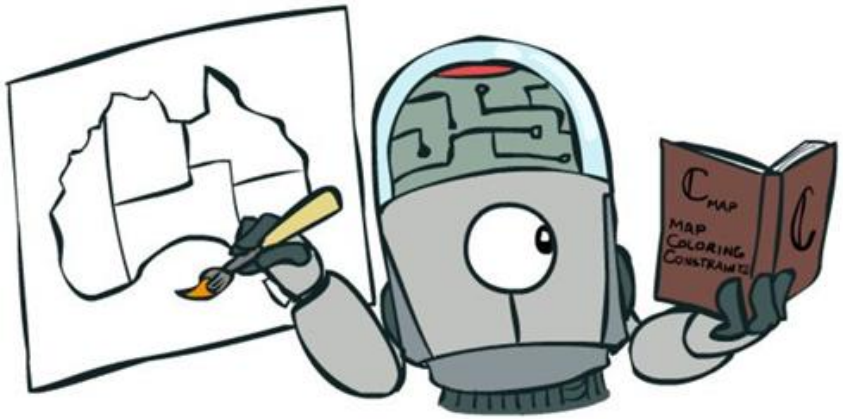
	Algorithm read inside a state.	Environment
Chapter 3	✗ black box	fully observable, deterministic, model known
4.3 Nondeterministic	✗	one action, several outcomes
4.4 Partially observable	✗	you cannot tell which state you are in
4.5 Online search	✗	the model itself is unknown
Chapter 5 CSP	✓ it reads it (factored)	same as Chapter 3

Constraint Satisfaction Problems

- Standard search problem:
  - State is a BLACK BOX: an arbitrary data structure.
  - Goal test can be any function over state
  - Successor function can also be anything



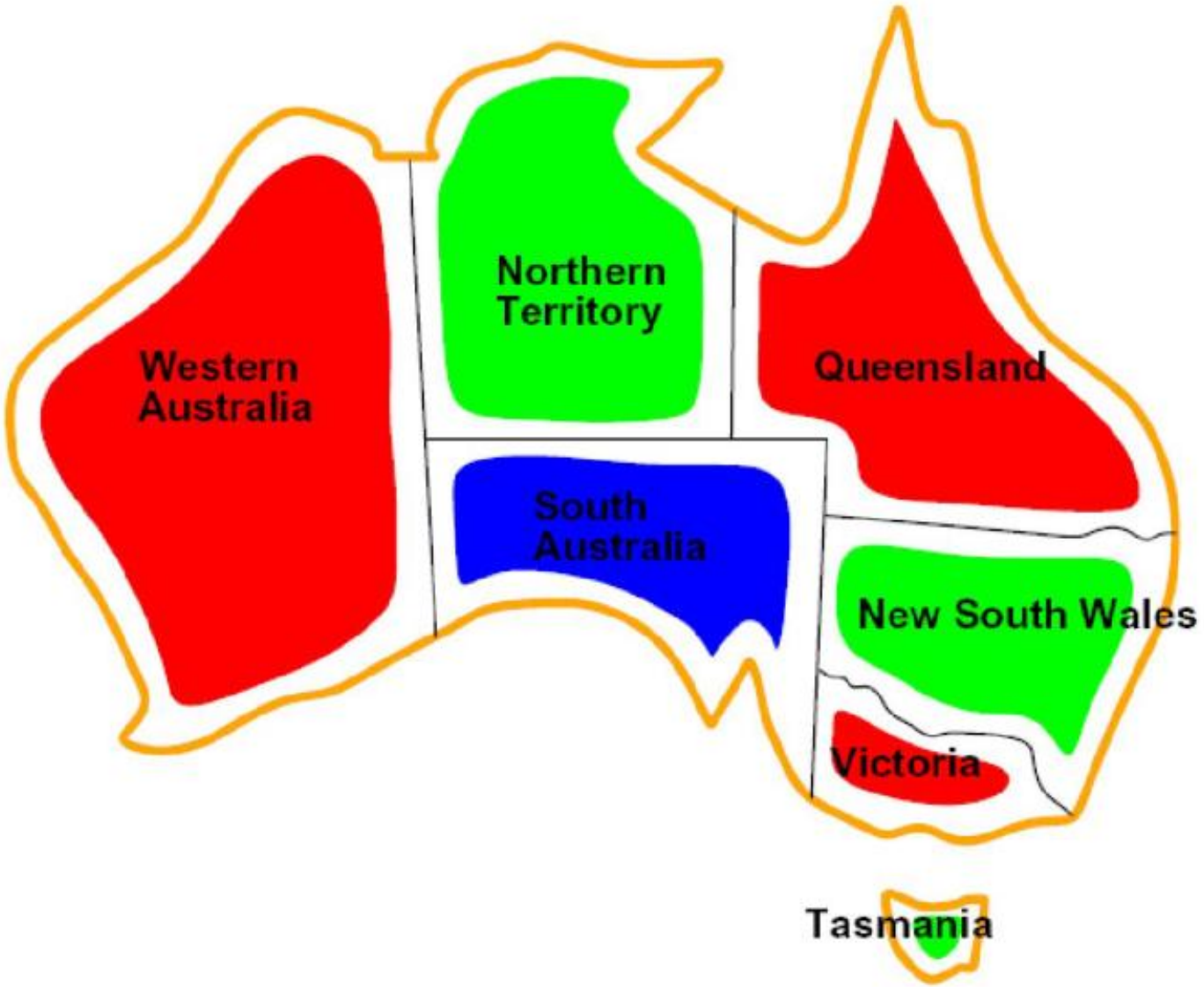
- Constraint satisfaction problems (CSPs):
  - A special subset of search problems.
  - State is defined by ***variables*** X_i with values from a ***domain*** D (sometimes D depends on i)
  - Goal test is a set of ***constraints*** specifying allowable combinations of values for subsets of variables



- Simple example of a formal representation language.

- Allows useful general-purpose algorithms with more power than standard search algorithms

Example: Map Coloring

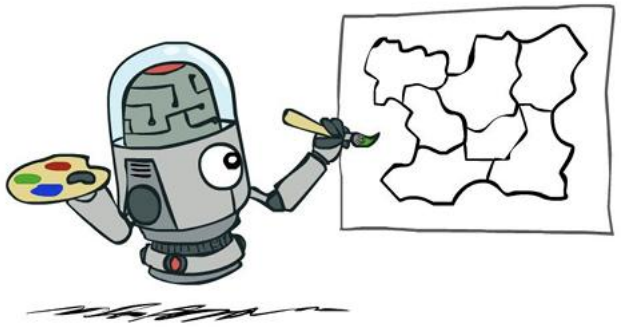


Example: Map Coloring

- Variables: one per region
  `WA, NT, Q, NSW, V, SA, T`
- Domains: `D = {red, green, blue}`
- Constraints: adjacent regions must take different colors
  - **Implicit**: that is a snippet of code - `WA ≠ NT`.
  - **Explicit:** it is the enumerated list of allowed pairs
    - `(WA,NT)∈{(red, green), (red, blue), …}`
- A solution is an assignment to every variable that satisfies every constraint

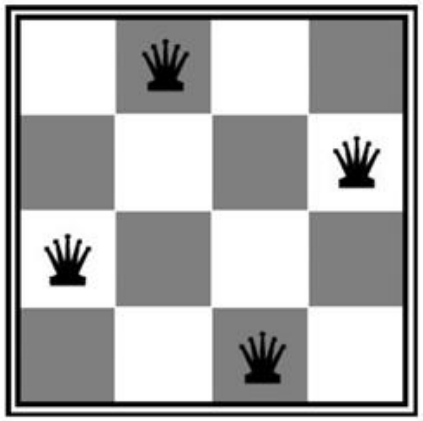
{WA=red, NT=green, Q=red, NSW=green,
V=red, SA=blue, T=green}





Example: N-Queens

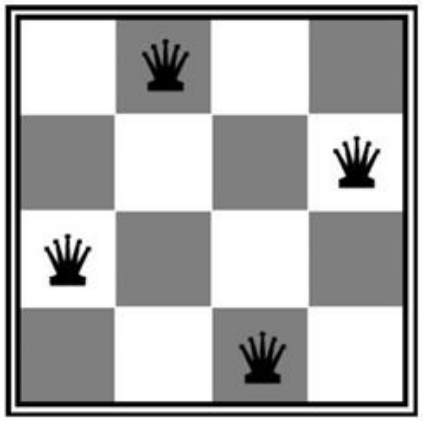
- Formulation 1
  - Variables:
  - Domains:
  - Constraints





Example: N-Queens

- Formulation 1
  - Variables: X_{ij}
  - Domains: $\{0, 1\}$
  - Constraints





$$\forall i, j, k \quad (X_{ij}, X_{ik}) \in \{(0,0), (0,1), (1,0)\}$$
$$\forall i, j, k \quad (X_{ij}, X_{kj}) \in \{(0,0), (0,1), (1,0)\}$$
$$\forall i, j, k \quad (X_{ij}, X_{i+k,j+k}) \in \{(0,0), (0,1), (1,0)\}$$
$$\forall i, j, k \quad (X_{ij}, X_{i+k,j-k}) \in \{(0,0), (0,1), (1,0)\}$$

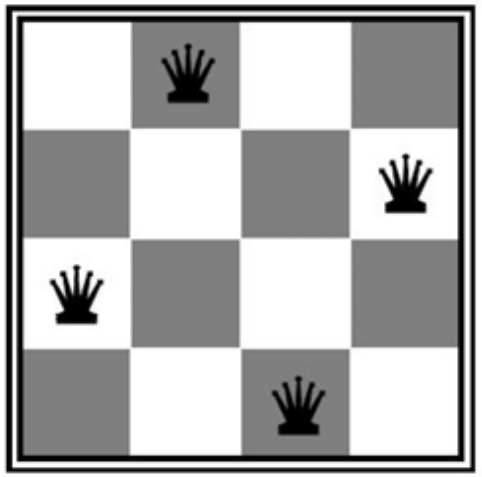
$$\sum_{i,j} X_{ij} = N$$

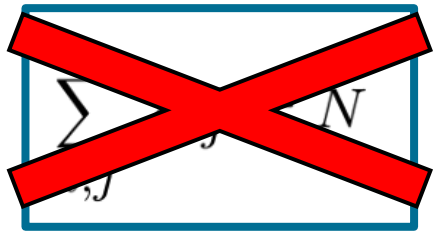
N squared area constraints.

Which is one explicit or implicit?...

Example: N-Queens (2)

- Formulation 2
  - Variables: Q_k
  - Domains: $\{11, 12, 13, \ldots 21, \ldots NN\}$
  - Constraints

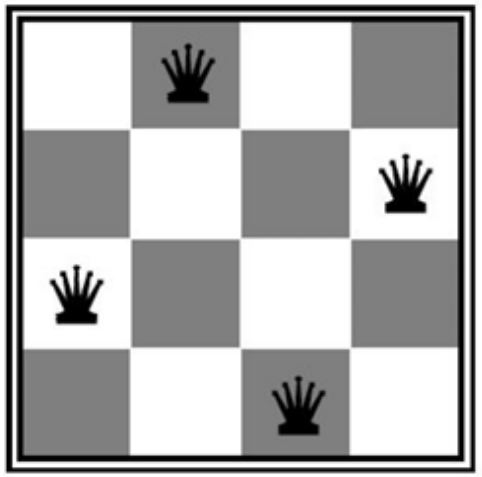




… there's an even better way! What is it?

Example: N-Queens (2)

- Formulation 2
  - Variables: Q_k
  - Domains: $\{11, 12, 13, \ldots 21, \ldots NN\}$
  - Constraints



$$\forall i, j \;\; \text{non-threatening}(Q_i, Q_j)$$

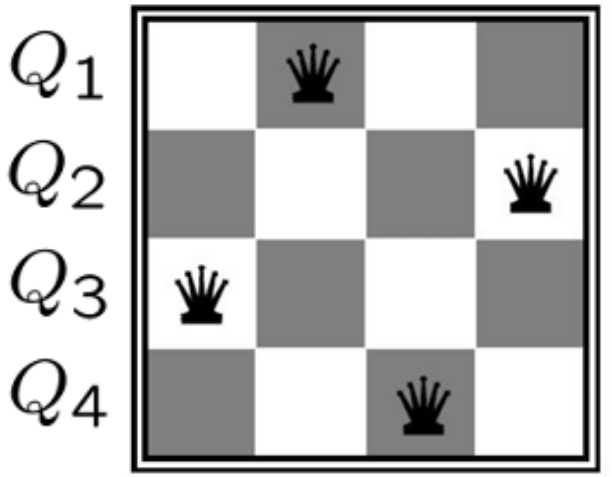
$$\forall i, j \;\; (Q_i, Q_j) \in \{(11, 23), (11, 24), \ldots\}$$

Which is one explicit or implicit?...

… there's an even better way! What is it?

Example: N-Queens (3)

- Formulation 3
  - Variables: Q_k
  - Domains: $\{1, 2, 3, \ldots N\}$
  - Constraints

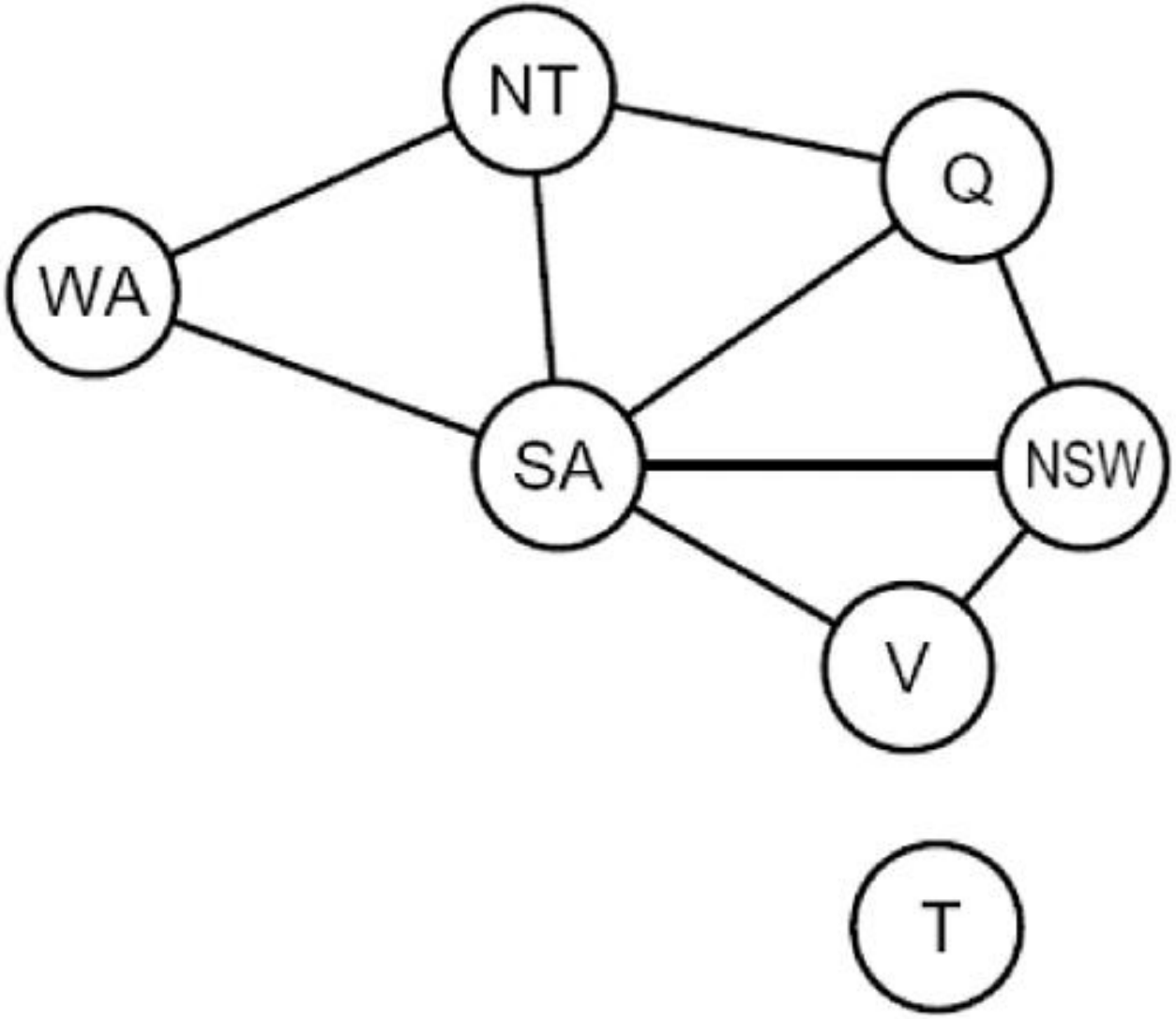


Implicit: $\forall i, j$ non-threatening(Q_i, Q_j)

Explicit: $(Q_1, Q_2) \in \{(1,3), (1,4), \ldots\}$

$\ldots$

3. Constraint Graphs

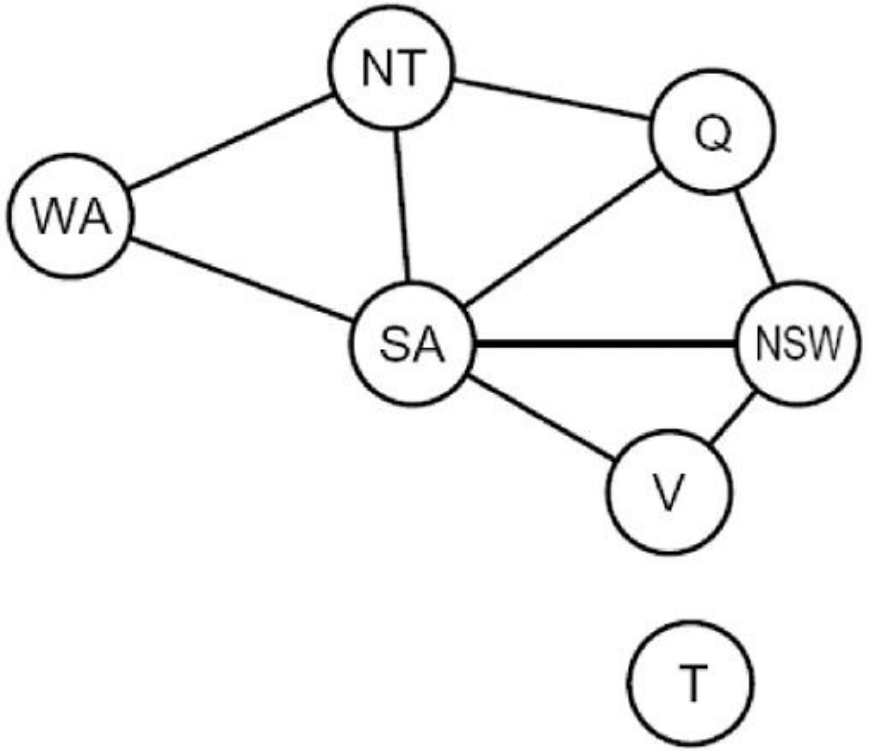


3. Constraint Graphs

- Binary CSP: each constraint relates (at most) two variables.
  ≠ Boolean CSP – how many values a domain holds

- Binary constraint graph: nodes are variables, edges show constraints.

- General-purpose CSP algorithms use the graph structure to speed up.
  E.g., Tasmania is an independent subproblem!



Example: Cryptarithmetic

- Formulation
  - Variables:
  - Domains:
  - Constraints:

```
  T W O
+ T W O
-------
F O U R
```



Example: Cryptarithmetic

- Formulation
  - Variables:

$$F\ T\ U\ W\ R\ O\ X_1\ X_2\ X_3$$

  - Domains:

$$\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$$

  - Constraints:

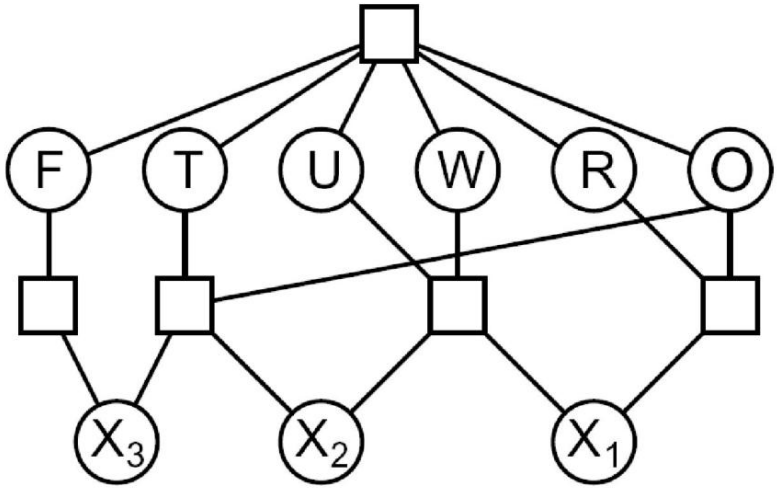
$$\text{alldiff}(F, T, U, W, R, O)$$

$$O + O = R + 10 \cdot X_1$$

$$\ldots$$

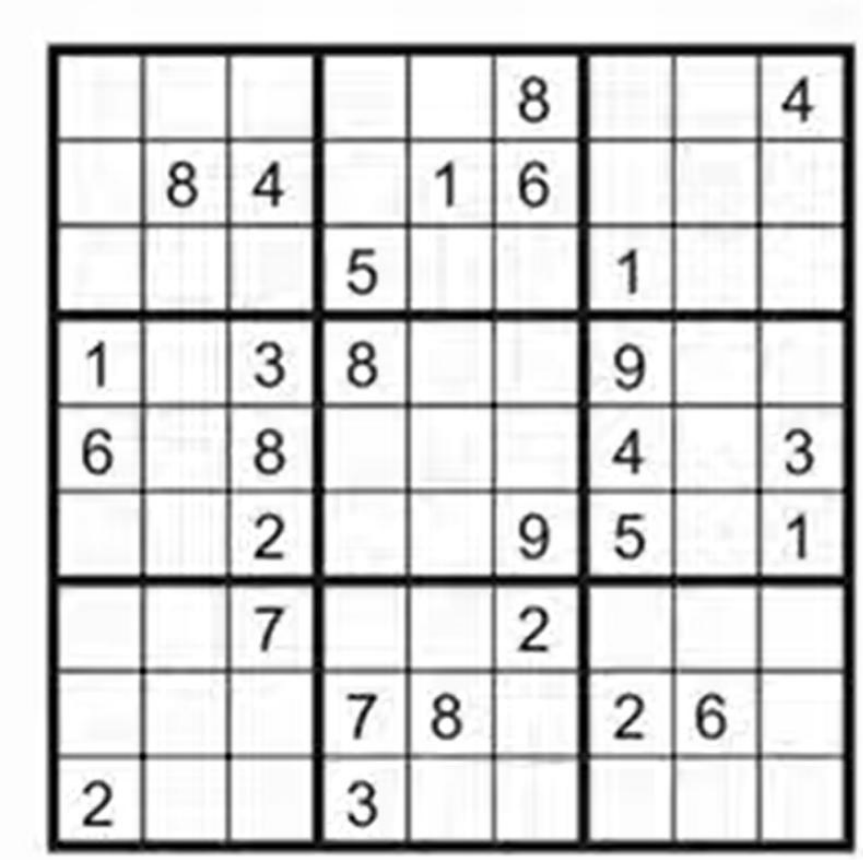
```
  T W O
+ T W O
-------
F O U R
```





Example: Sudoku

- Formulation
  - Variables: Each (open) square
  - Domains:
  - Constraints:



					8			4
	8	4		1	6			
			5			1		
1		3	8			9		
6		8				4		3
		2			9	5		1
		7			2			
			7	8		2	6	
2			3					

Example: Sudoku

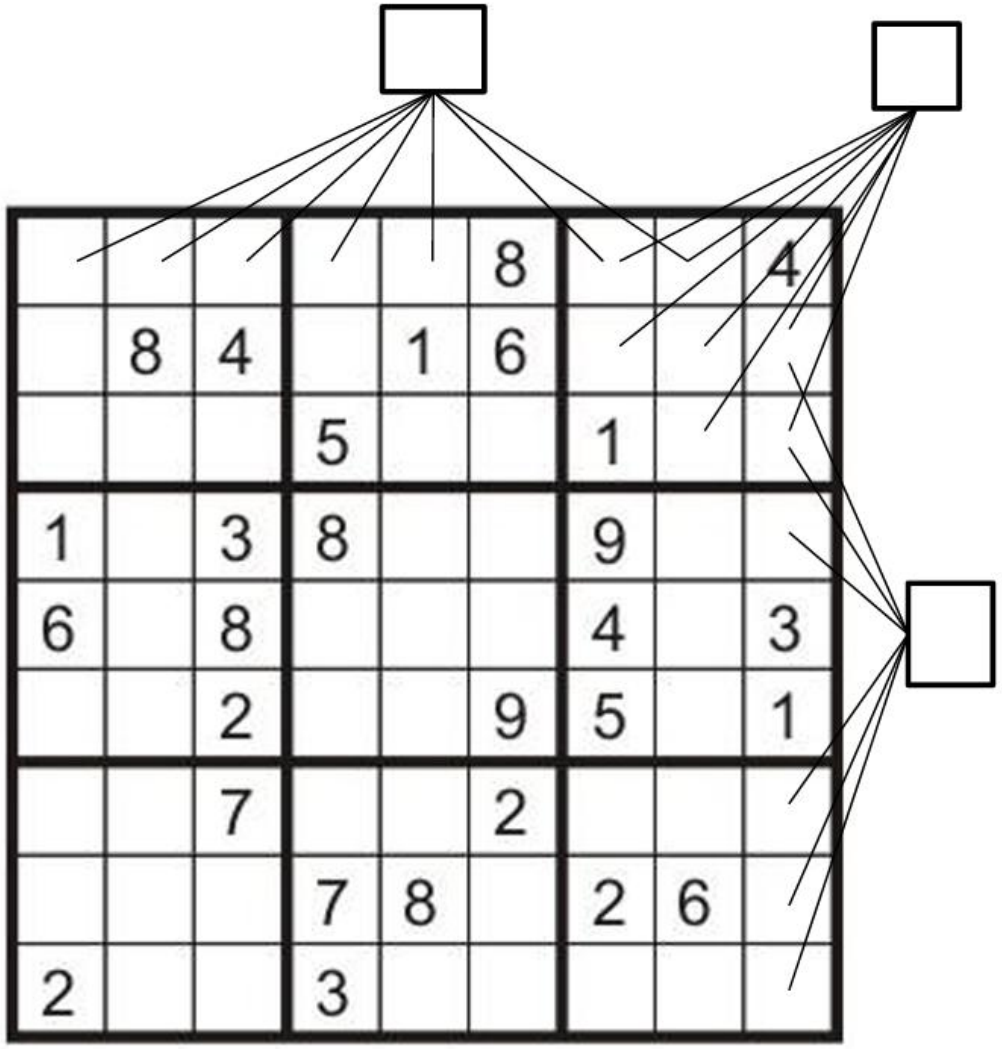
- Formulation
  - Variables: Each (open) square
  - Domains: {1,2,…,9}
  - Constraints: 9-way alldiff for each column

    9-way alldiff for each row

    9-way alldiff for each region

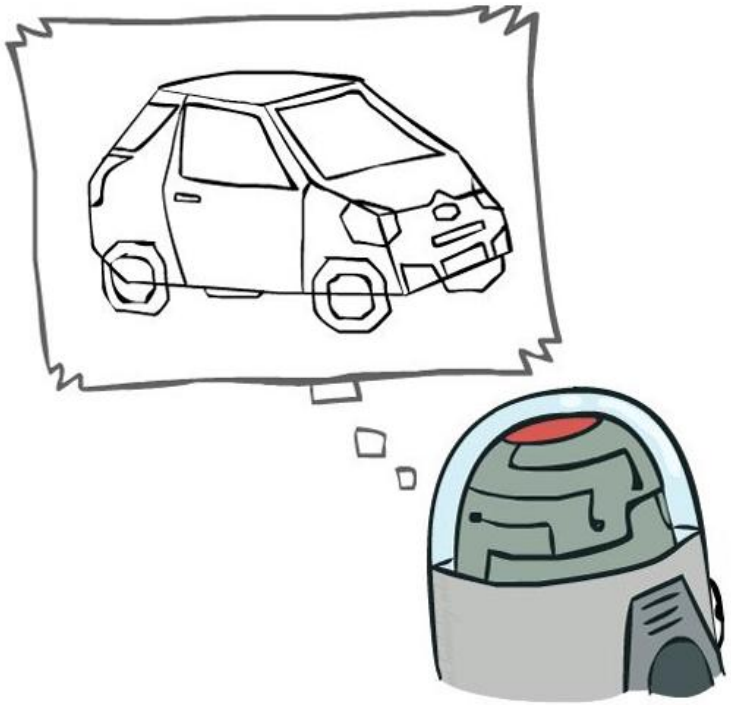
    (or can have a bunch of pairwise inequality constraints)

    Some squares already filled in (unary constraints)

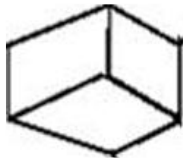


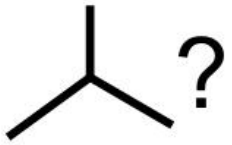
Example: The Waltz Algorithm

- The Waltz algorithm is for interpreting line drawings of solid polyhedra as 3D objects
- An early example of an AI computation posed as a CSP









- Approach:
  - Each intersection is a variable
  - Adjacent intersections impose constraints on each other
  - Solutions are physically realizable 3D interpretations

Introduction to Artificial Intelligence

Chapter 5 – Part II: Search CSP

Soongsil University



Source: UC Berkeley CS188, slides by Dan Klein and Pieter Abbeel (ai.berkeley.edu)

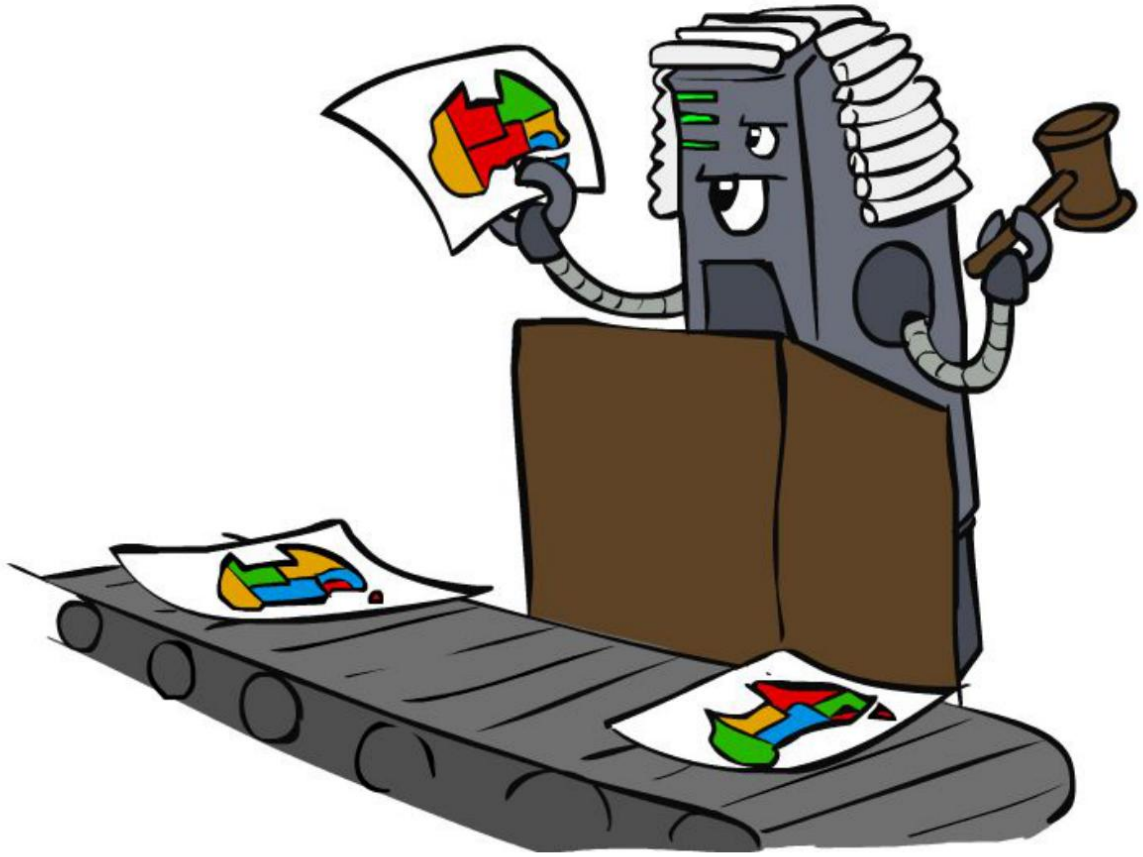
4. Solving CSPs



Standard Search Formulation

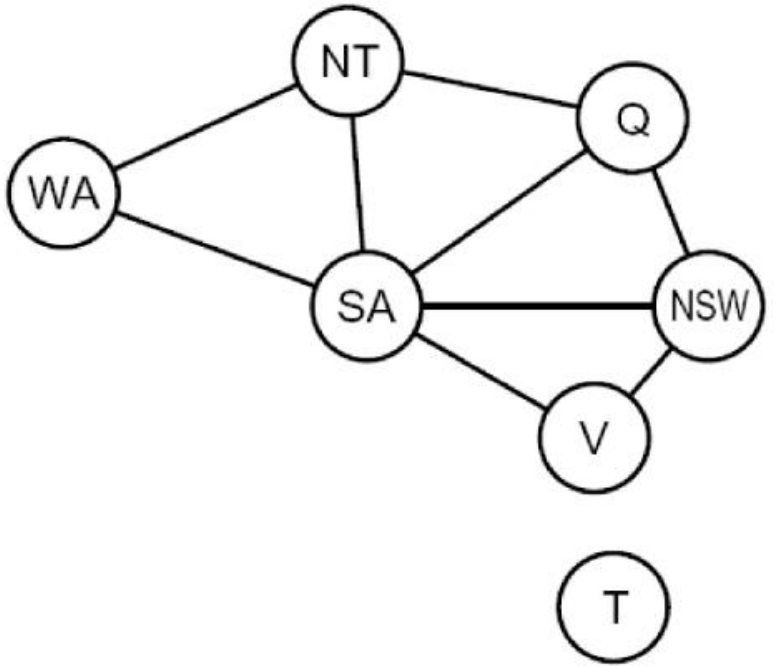
- Standard search formulation of CSPs
  - States: the values assigned so far (partial assignments)
  - Initial state: the empty assignment, {}
  - Successor function: assign a value to an unassigned variable
  - Goal test: the current assignment is complete and satisfies all constraints
  - Path cost: ???

- We'll start with the straightforward, naïve approach, then improve it.



Search Methods

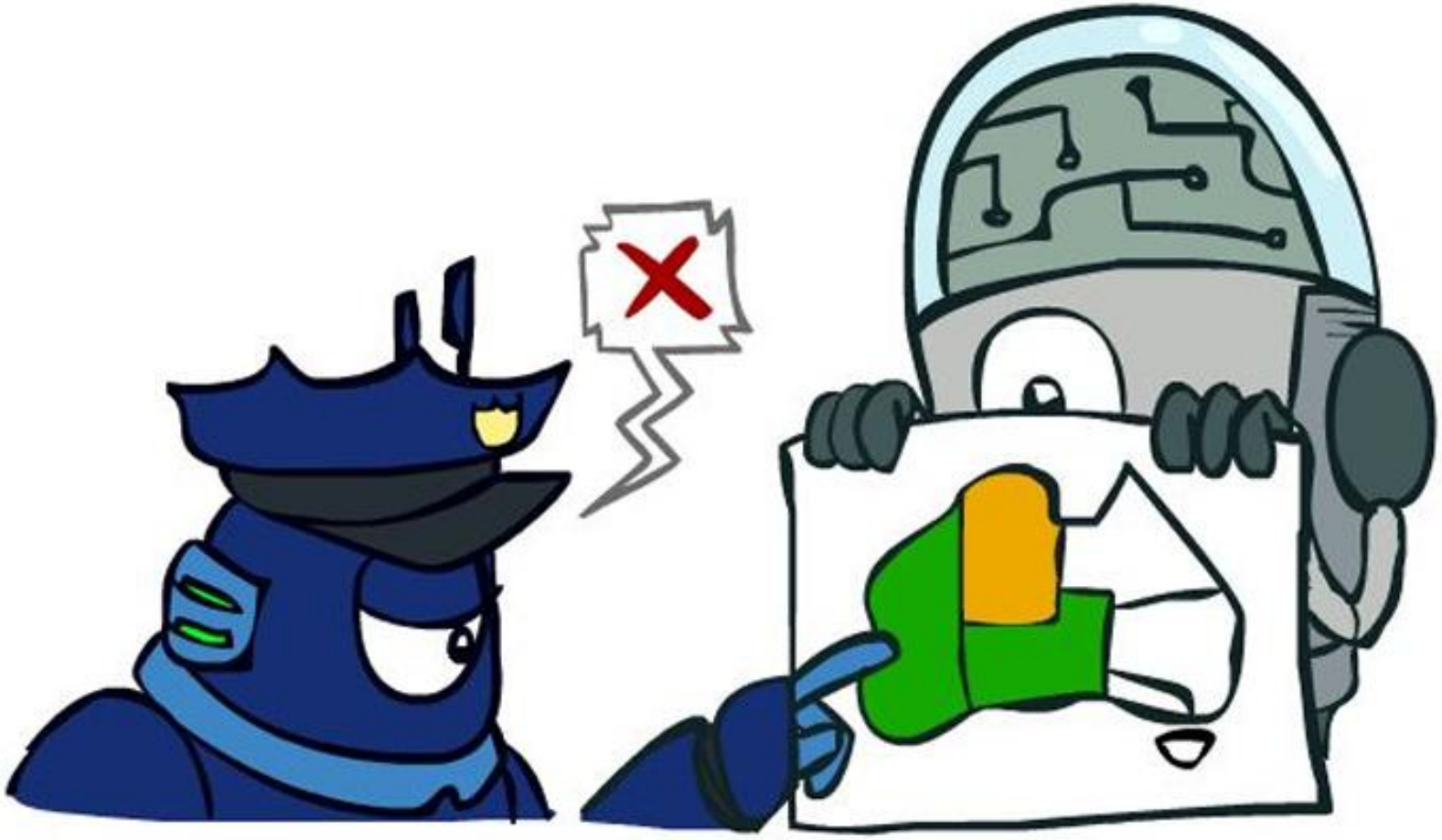
- What would BFS do?
  - Root: Empty assignment: {}
  - Sweep over partial assignments to 1 variable:
    `{WA = R}, {WA = B}, {WA = G},`
    `{NT = R}…`
  - Then partial assignments to 2 variables:
    `{WA = R, NT = R}, {WA = R, NT = B}…`
  - …has to go through whole tree.
  - All solutions are at the bottom = bad!



- What would DFS do?
  - Perhaps more reasonable? At least it starts by checking complete solutions
  - In CSPs, It seems like IDS.

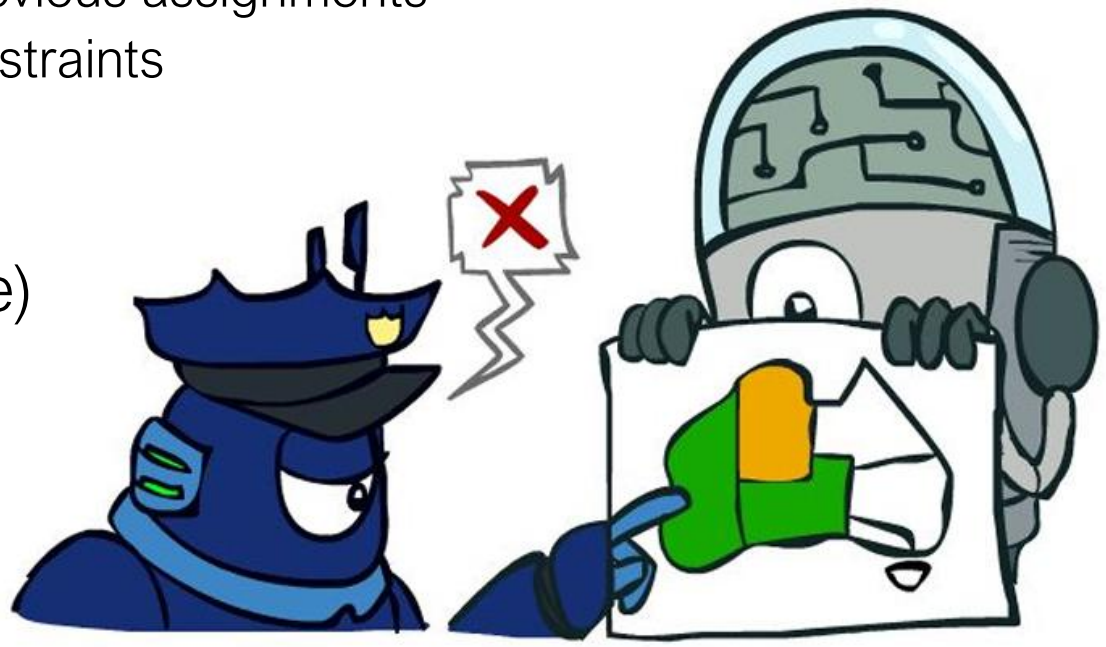


Backtracking Search

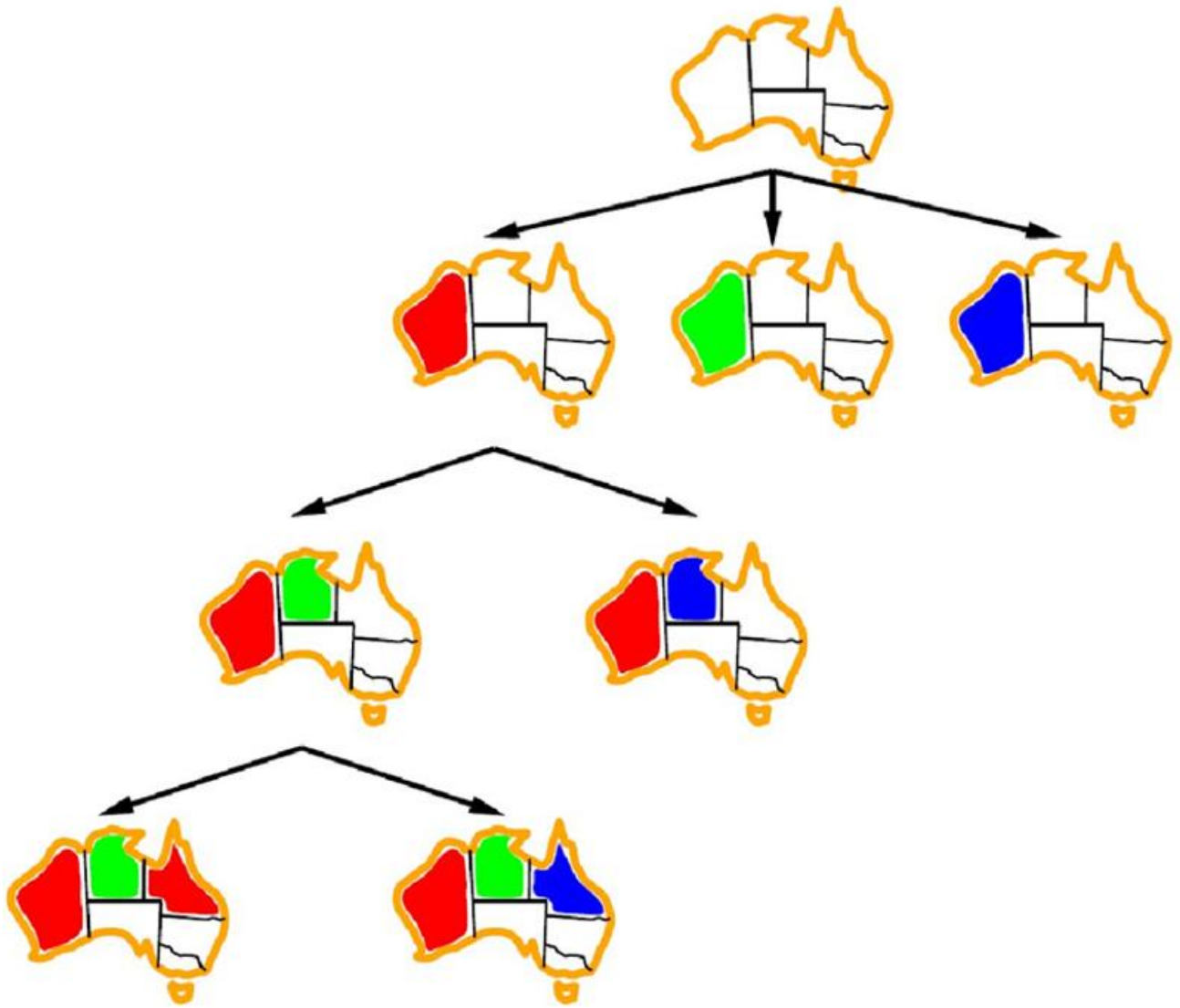


Backtracking Search

- Backtracking search is the basic uninformed algorithm for solving CSPs
- Idea 1: One variable at a time
  - Variable assignments are commutative, so fix ordering
  - I.e., [WA = red then NT = green] same as [NT = green then WA = red]
  - Only need to consider assignments to a single variable at each step
- Idea 2: Check constraints as you go
  - I.e. consider only values which do not conflict with previous assignments
  - Might have to do some computation to check the constraints
  - "Incremental goal test"
- Depth-first search with these two improvements is called ***backtracking search*** (not the best name)
- Can solve n-queens for n ≈ 25

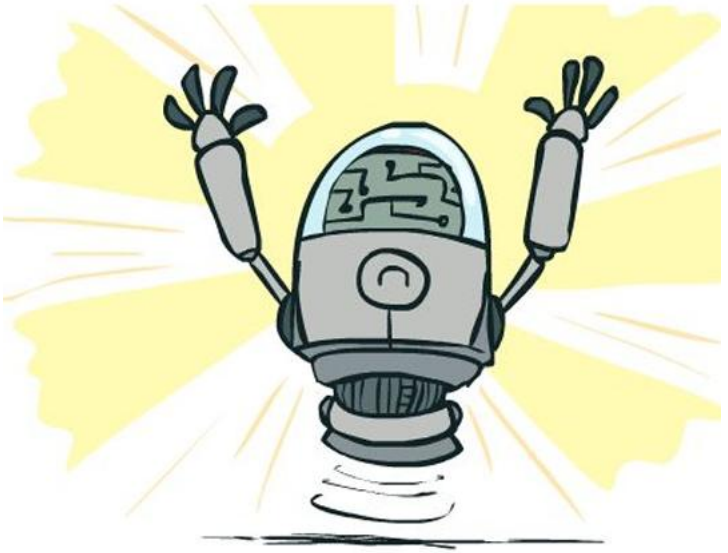


Backtracking Example



Improving Backtracking

- General-purpose ideas give huge gains in speed
  - (Unlike A* heuristics, these apply across many problems)



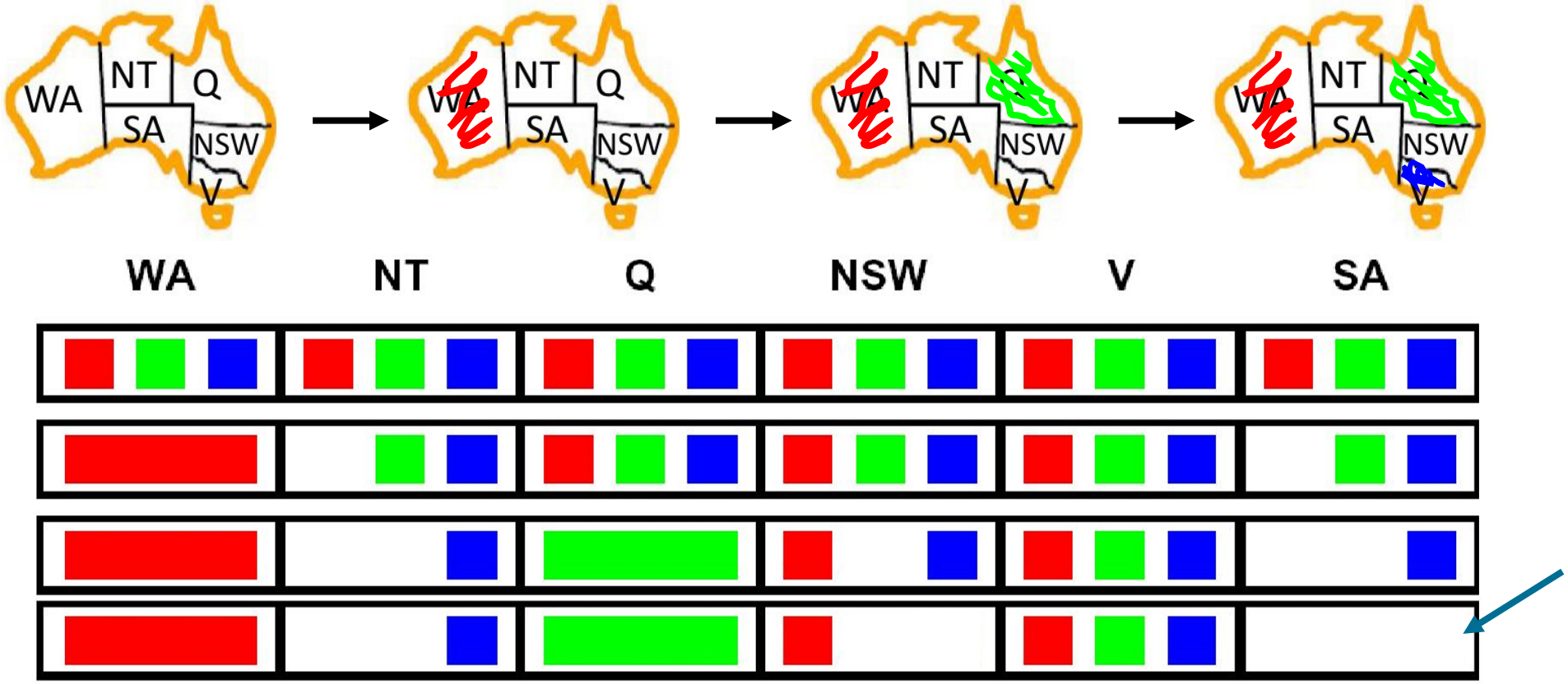
- Filtering: Can we detect inevitable failure early?

- Ordering:
  - Which variable should be assigned next?
  - In what order should its values be tried?

- Structure: Can we exploit the problem structure?

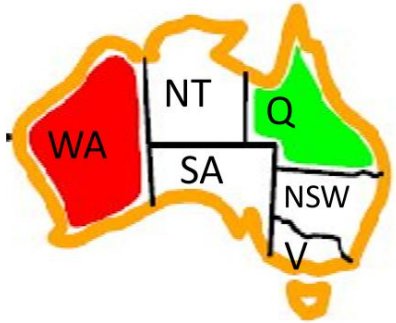
Filtering: Forward Checking

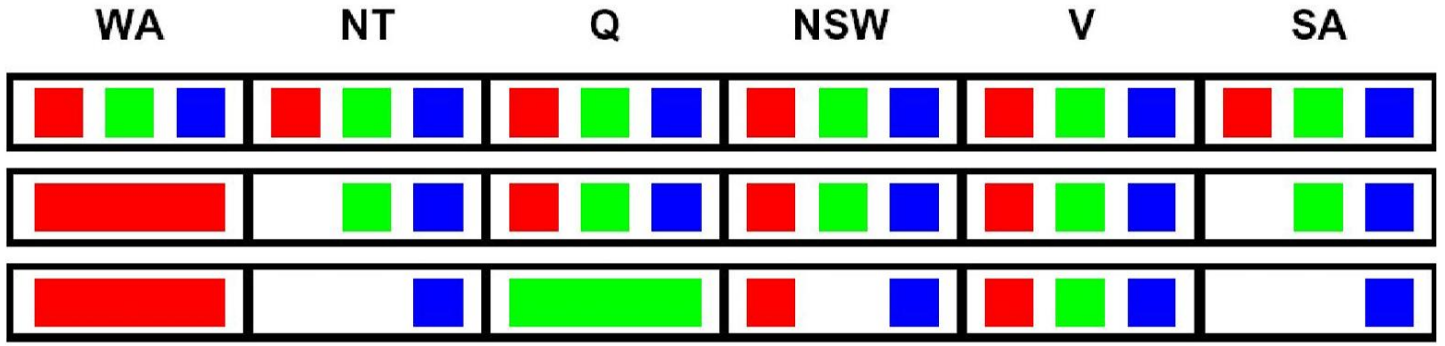
- Filtering: Keep track of domains for unassigned variables and remove bad options
- Forward checking: Remove values that violate a constraint when added to the existing assignment



Constraint Propagation in Forward Checking

- Forward checking propagates information from assigned to unassigned variables, but doesn't provide early detection for all failures:

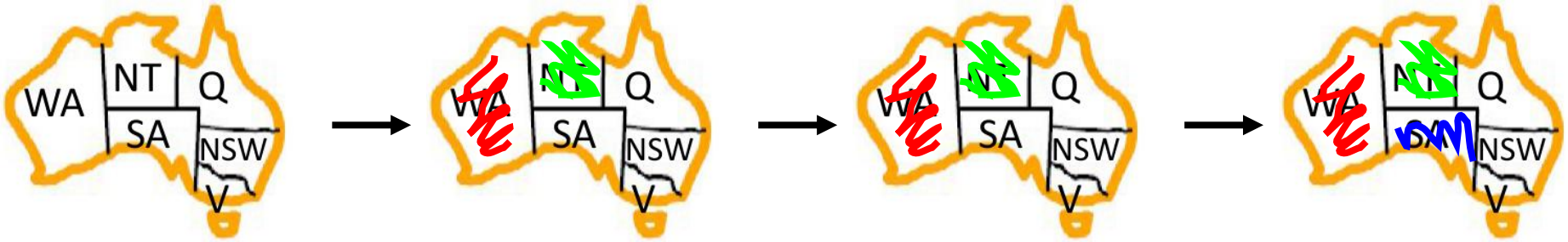




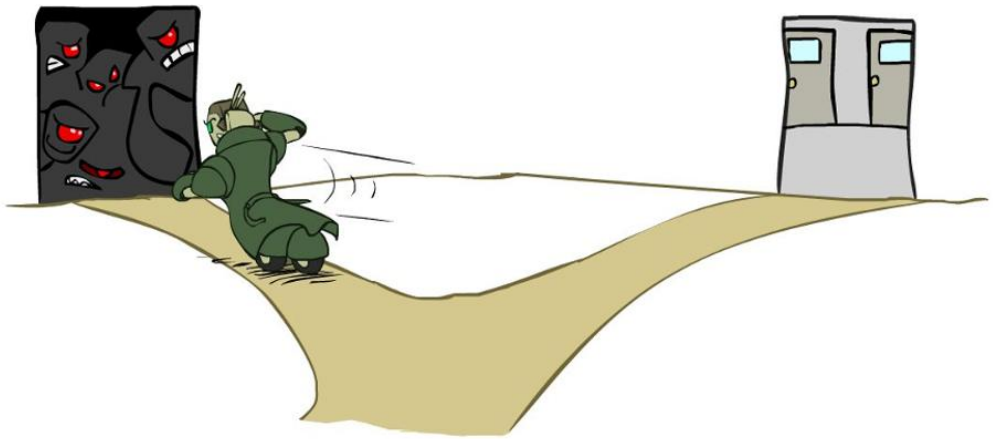
- NT and SA cannot both be blue!
- Why didn’t we detect this yet?
- **Constraint propagation**: reason from constraint to constraint

Ordering: (1) Minimum Remaining Values

- **Variable Ordering** - Minimum remaining values (MRV):
  - Choose the variable with the fewest legal left values in its domain



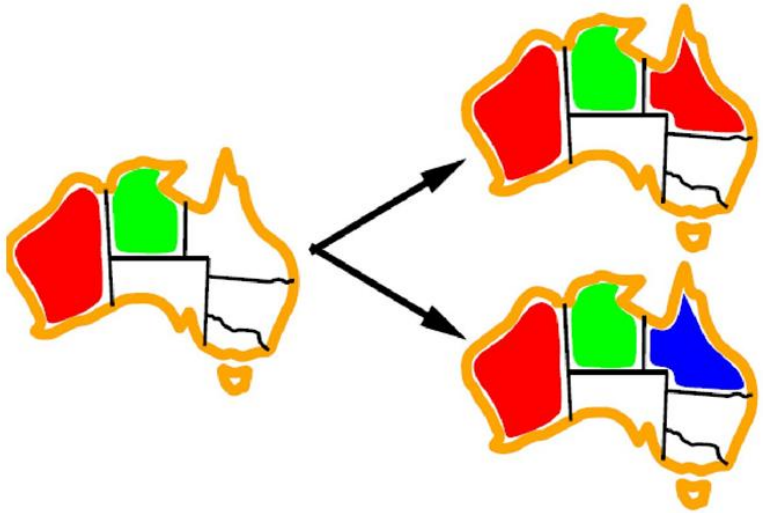
- Why min rather than max?
  - Must assign every **variable** eventually.
  - **"not which to do"** but **"when to find out it fails"**



- Also called "***most constrained variable***" or **"Fail-fast" ordering.**

Ordering: (2) Least Constraining Value

- **Value Ordering**: Least Constraining Value
  - Given a choice of variable, choose the least constraining value
  - I.e., the one that rules out the fewest values in the remaining variables
  - Note that it may take some computation to determine this! (E.g., rerunning filtering)



- Why least rather than most?
  - Must assign every VARIABLE eventually.
  - do NOT have to try all the VALUES.

